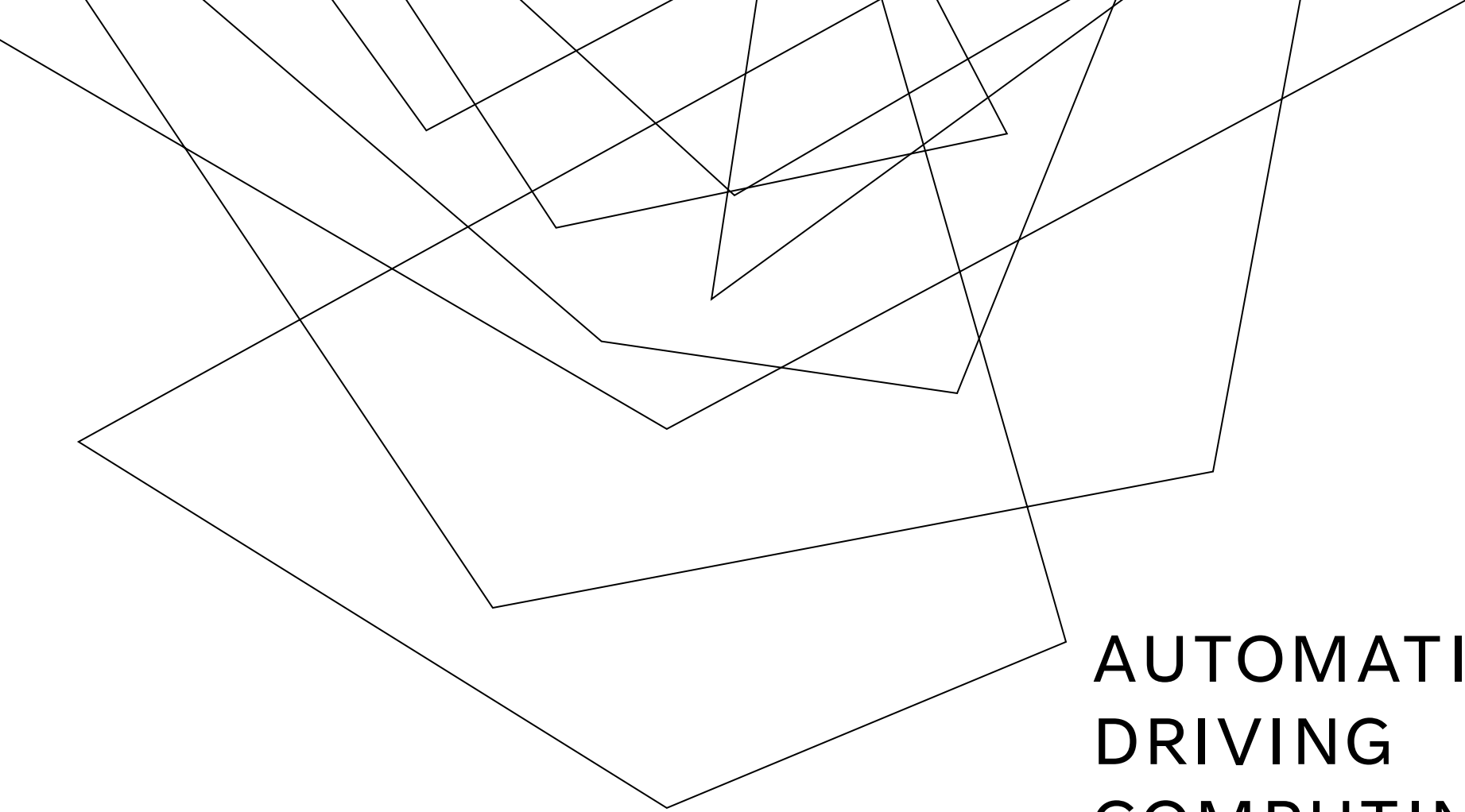


A series of thin, black, overlapping lines forming various geometric shapes like triangles and polygons, creating a complex, abstract pattern in the upper left portion of the slide.

AUTOMATIC DRIVING COMPUTING

4. PERCEPTION (2)





지난 시간 복습

VISION BACKBONE

1. Perception – overview
2. Image classification
3. Convolutional neural network
4. Vision transformer





목차

목차

1. Perception – overview
2. Object localization
3. Object detection
4. R-CNN

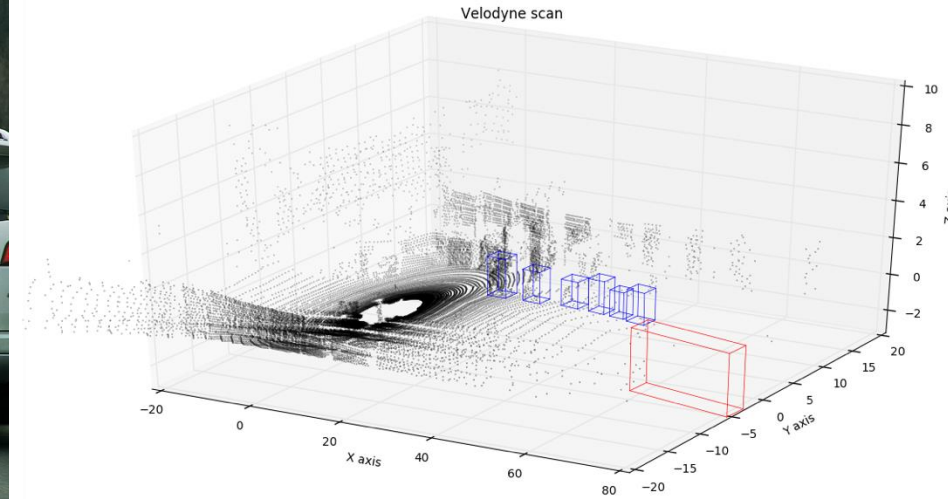


An abstract graphic consisting of two thin, dark grey lines intersecting on a light grey background. One line runs diagonally from the top-left towards the bottom-right, and the other runs from the top-right towards the bottom-left, forming an 'X' shape.

PERCEPTION - OVERVIEW

PERCEPTION

Sensor 데이터를 가지고 주위의 물체들을 어떻게 식별 (Perception) 할까?



PERCEPTION

Sensor 데이터를 가지고 주위의 물체들을 어떻게 식별 (Perception) 할까?

- 1) Camera (image) 데이터 먼저
- 2) 단일 물체만 포함되어 있는 이미지 먼저 (= Image classification)
(Divide and conquer!)



PERCEPTION

Sensor 데이터를 가지고 주위의 물체들을 어떻게 식별 (Perception) 할까?

3) 단일 물체 Object localization

4) 다중 물체 Object detection

① 분류
(Classification)



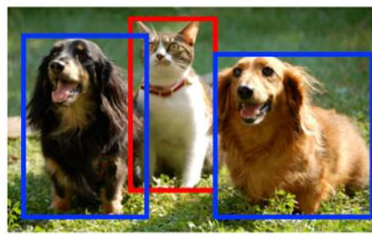
고양이

② 분류
+ Localization



고양이

③ 객체 탐지
(Object Detection)



고양이, 개

④ 세그멘테이션
(Segmentation)



고양이, 개

객체 하나

여러 객체



An abstract graphic design featuring two thin, dark gray lines that intersect on a light gray background. One line runs diagonally from the top-left towards the bottom-right, while the other runs from the top-right towards the bottom-left. The intersection point is located to the left of the text.

OBJECT LOCALIZATION

TASK DEFINITION

Input : 단일 물체가 담긴 RGB 이미지

Output : 이미지 내 물체의 category label (“What”)
& 위치 (“Where”)



TASK DEFINITION

Input : 단일 물체가 담긴 RGB 이미지

Output : 이미지 내 물체의 category label (“What”)
& 위치 (“Where”)

물체의 위치를 결정하기 위해 필요한 숫자 : 4개

(x, y, w, h) 혹은 (x1, y1, x2, y2) - “bounding box”



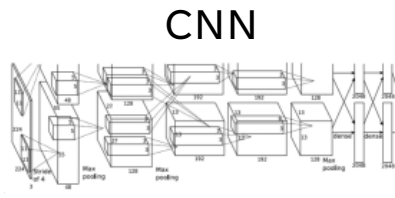
MODEL TRAINING PIPELINE

(1) Classification part : Cross entropy loss

Detecting a single object “What”



[This image is CC0 public domain](#)



CNN

Vector:
4096

Fully
Connected:
4096 to 1000

Class Scores

Cat: 0.9
Dog: 0.05
Car: 0.01
...

Correct label:

Cat

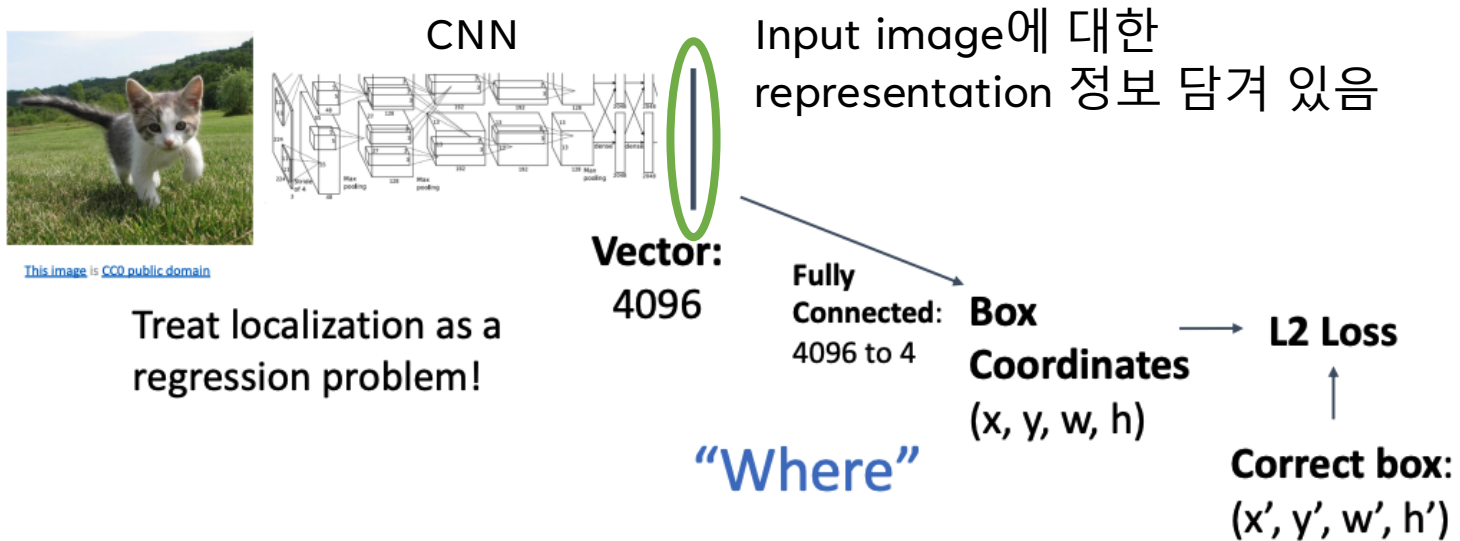
↓
**Softmax
Loss**



MODEL TRAINING PIPELINE

(2) Localization part : L2 loss

Detecting a single object



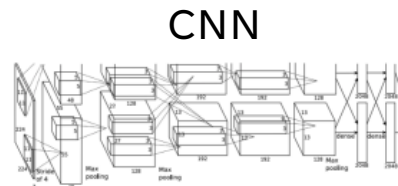
MODEL TRAINING PIPELINE

두개의 loss 동시에 사용하여 모델 학습

Detecting a single object “What”



[This image is CC0 public domain](#)



Treat localization as a regression problem!

Vector:
4096

Fully
Connected:
4096 to 1000

Class Scores

Cat: 0.9
Dog: 0.05
Car: 0.01
...

Correct label:
Cat

Softmax

Loss

**Multitask
Loss**

**Weighted
Sum**

Loss

$$L = L_{cls} + \lambda L_{reg}$$

Fully
Connected:
4096 to 4

**Box
Coordinates**
(x, y, w, h)

L2 Loss

Correct box:
(x', y', w', h')

“Where”



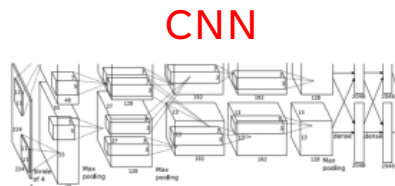
MODEL TRAINING PIPELINE

Problem: bounding box는 라벨링이 어려움 → 데이터셋 크기 확장이 어려움

Detecting a single object “What”



[This image is CC0 public domain](#)



CNN

Treat localization as a regression problem!

Vector:
4096

Fully
Connected:
4096 to 1000

Class Scores

Cat: 0.9
Dog: 0.05
Car: 0.01
...

Correct label:
Cat

Softmax
Loss

Multitask
Loss

Weighted
Sum → Loss

$$L = L_{cls} + \lambda L_{reg}$$

Fully
Connected:
4096 to 4

Box
Coordinates
(x, y, w, h)

L2 Loss

Correct box:
(x', y', w', h')

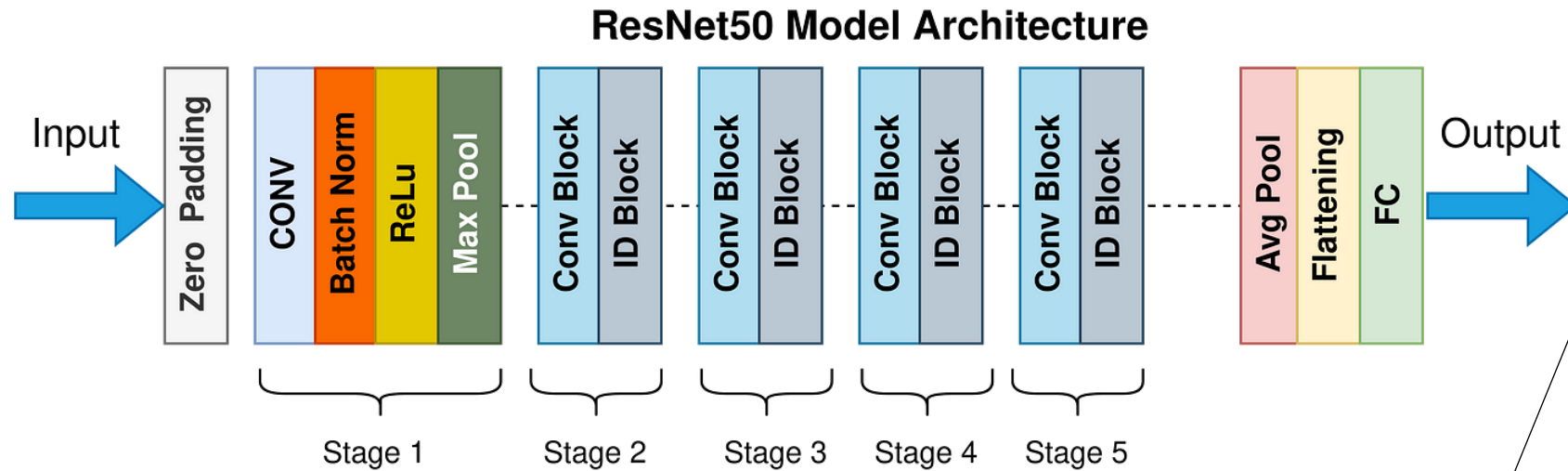
“Where”



TRANSFER LEARNING

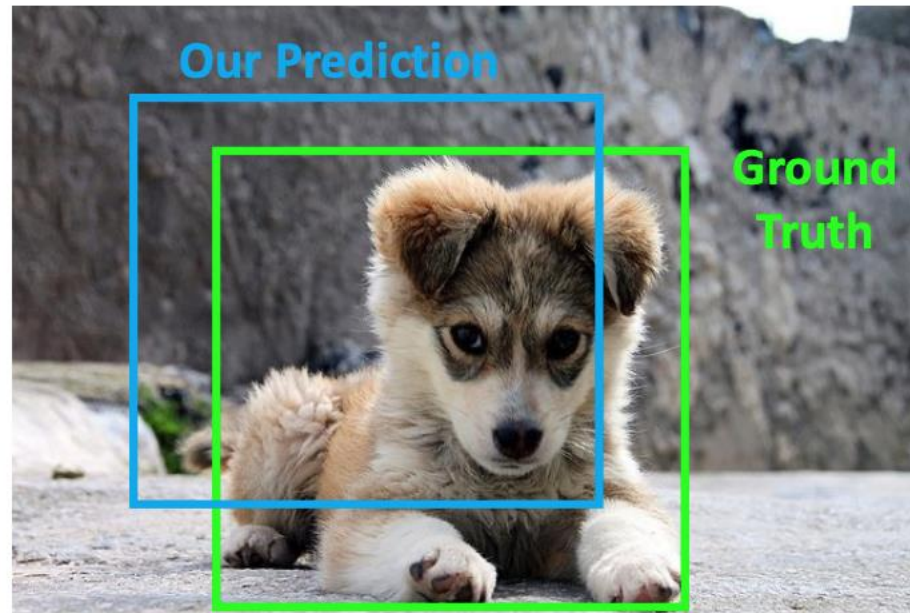
Solution : ImageNet dataset, image classification task로 학습한 CNN 모델의 weight를 가져와서 사용하자!

Cf)



INTERSECTION OVER UNION

Q) 모델 학습 후 Bounding box prediction의 성능을 어떻게 평가?

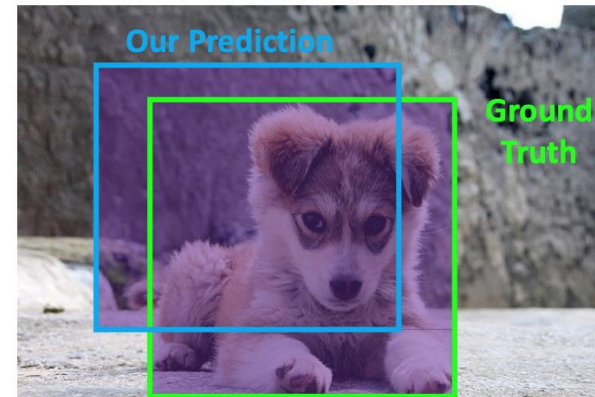
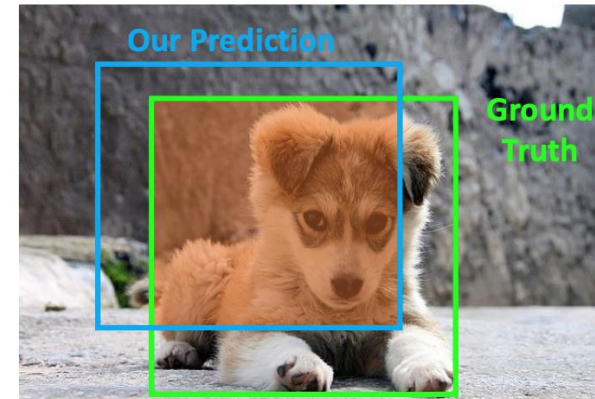


INTERSECTION OVER UNION

Q) 모델 학습 후 Bounding box prediction의 성능을 어떻게 평가?

A) Intersection (교집합) over Union (합집합) → IoU

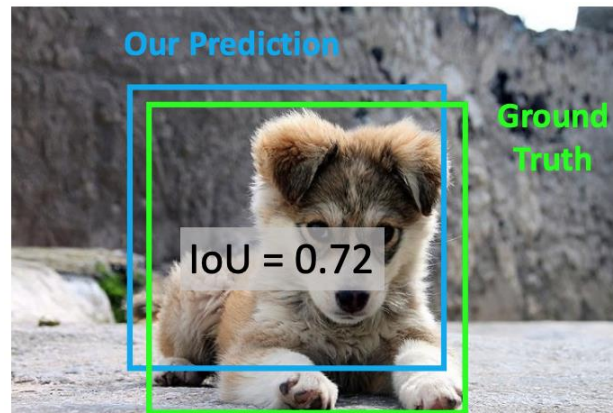
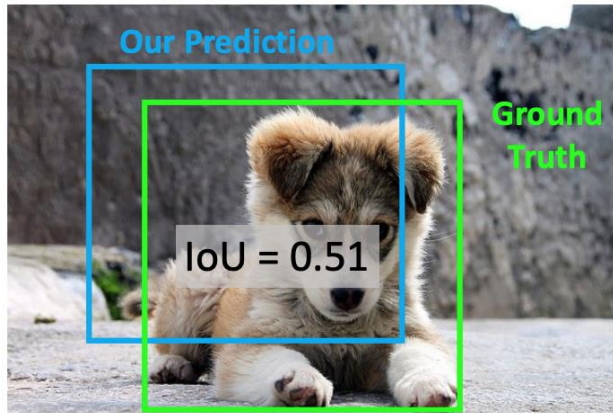
$$\frac{\text{Area of Intersection}}{\text{Area of Union}}$$



INTERSECTION OVER UNION

Q) 모델 학습 후 Bounding box prediction의 성능을 어떻게 평가?

A) Intersection (교집합) over Union (합집합) → IoU





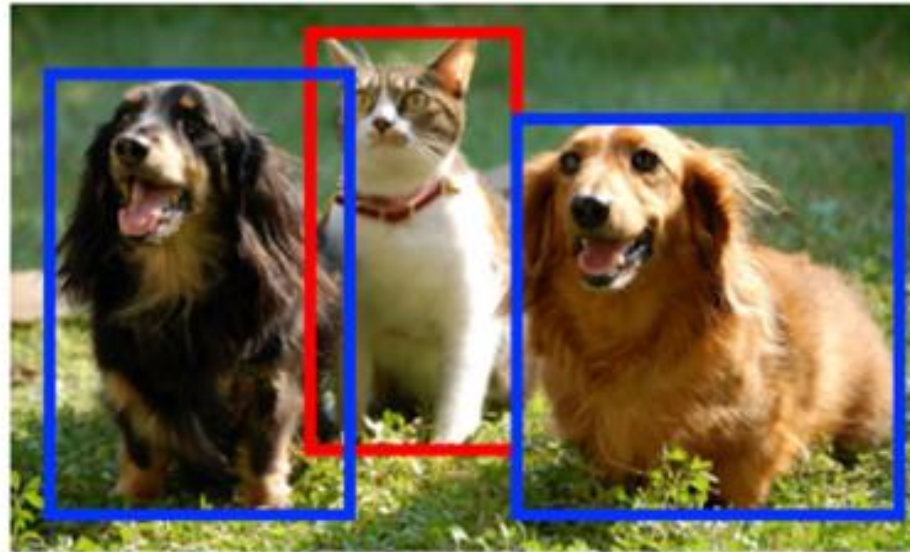
OBJECT
DETECTION

The image features a minimalist design on a light gray background. Two thin, dark gray lines intersect: one line runs diagonally from the top-left towards the bottom-right, and the other runs from the top-right towards the bottom-left. To the right of this intersection, the words 'OBJECT' and 'DETECTION' are stacked vertically in a bold, black, sans-serif typeface.

TASK DEFINITION

Input : **다중** 물체가 담긴 RGB 이미지

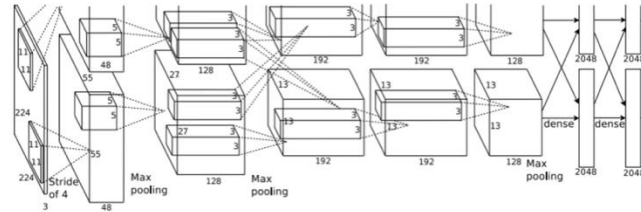
Output : 이미지 내 **모든 물체 각각에 대한** category label (“What”)
& 위치 (“Where”)



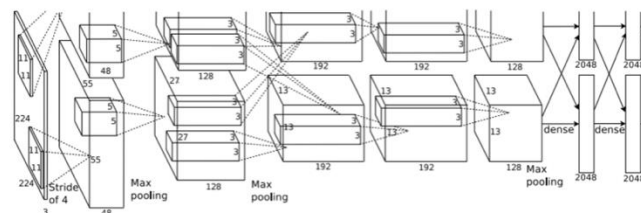
TASK DEFINITION

Problem 1) 입력으로 들어오는 이미지마다 물체의 개수가 다름

필요한 Class label & bounding box의 개수가 입력 이미지마다 다름



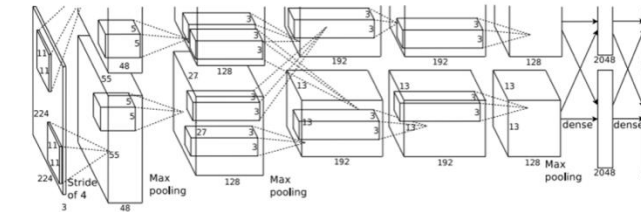
CAT: (x, y, w, h)



DOG: (x, y, w, h)

DOG: (x, y, w, h)

CAT: (x, y, w, h)



DUCK: (x, y, w, h)

DUCK: (x, y, w, h)

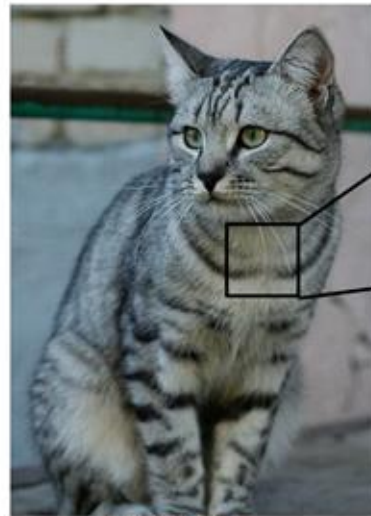
....



TASK DEFINITION

Problem 2) 사람은 이미지에서 물체가 몇 개 있는지 보면 바로 알지만
컴퓨터는 그렇지 않음

The Problem: Semantic Gap



```
[[105 112 100 111 104 99 106 99 96 103 112 119 104 97 93 87]
[ 91 98 102 106 104 79 98 103 99 105 123 116 110 105 94 85]
[ 76 85 90 105 120 105 87 96 95 99 115 112 106 103 99 85]
[ 99 81 81 93 120 131 127 100 95 98 102 99 96 93 101 94]
[106 91 61 64 69 91 88 85 101 107 109 98 75 84 96 95]
[114 108 85 55 55 69 64 54 64 87 112 129 90 74 84 91]
[133 137 147 103 65 81 88 65 52 54 74 84 102 93 85 82]
[128 137 144 140 99 95 86 78 62 65 63 63 68 73 86 101]
[125 133 140 137 119 121 117 94 65 79 88 65 54 64 72 98]
[127 125 131 147 133 127 126 131 111 96 89 75 61 64 72 84]
[115 114 109 123 150 140 131 118 113 109 100 92 74 65 72 78]
[ 89 93 98 97 100 147 131 118 113 114 113 109 106 95 77 80]
[ 63 77 86 81 77 79 102 123 117 115 117 125 125 130 115 87]
[ 62 65 82 89 78 71 88 101 124 126 119 101 107 114 131 119]
[ 63 65 75 88 89 71 62 81 120 138 135 105 81 98 110 118]
[ 87 65 71 87 106 95 69 45 76 130 126 107 92 94 105 112]
[110 97 82 86 117 123 116 66 41 51 95 93 89 95 102 107]
[164 146 112 88 82 120 124 104 76 48 45 66 88 101 102 109]
[157 170 157 120 93 86 114 132 112 97 69 55 70 82 99 94]
[130 120 134 161 139 100 109 118 121 134 114 87 65 53 69 86]
[120 112 96 117 150 144 120 115 104 107 102 93 87 81 72 79]
[123 107 96 96 83 112 153 149 122 109 104 75 88 107 112 99]
[122 121 102 88 82 86 94 117 145 140 153 102 58 70 92 107]
[122 164 140 103 71 56 78 83 93 103 119 139 102 61 69 84]]
```

What the computer sees

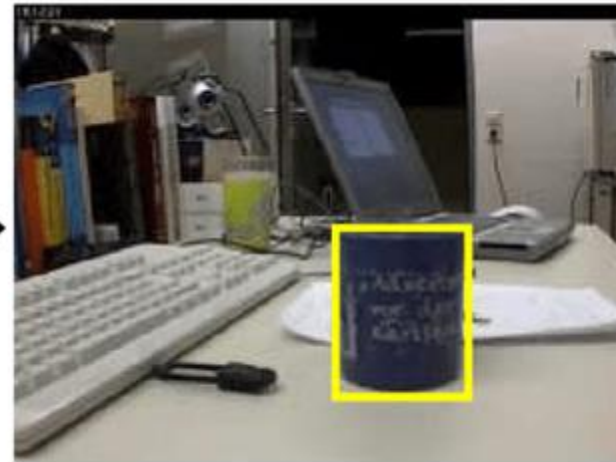
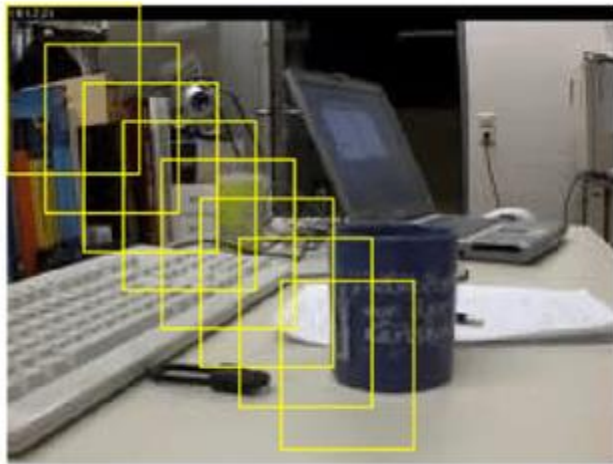
An image is a tensor of integers
between [0, 255]:

e.g. 800 x 600 x 3
(3 channels RGB)



TASK DEFINITION

Problem 3) 물체가 존재할 수 있는 위치 및 물체의 크기에 대한
모든 경우의 수는 매우 많음



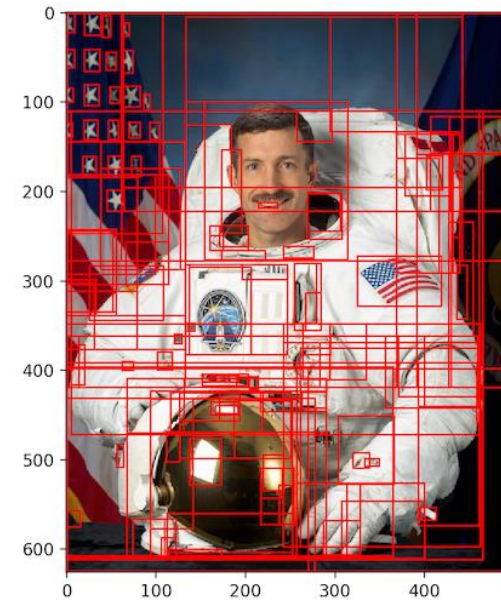
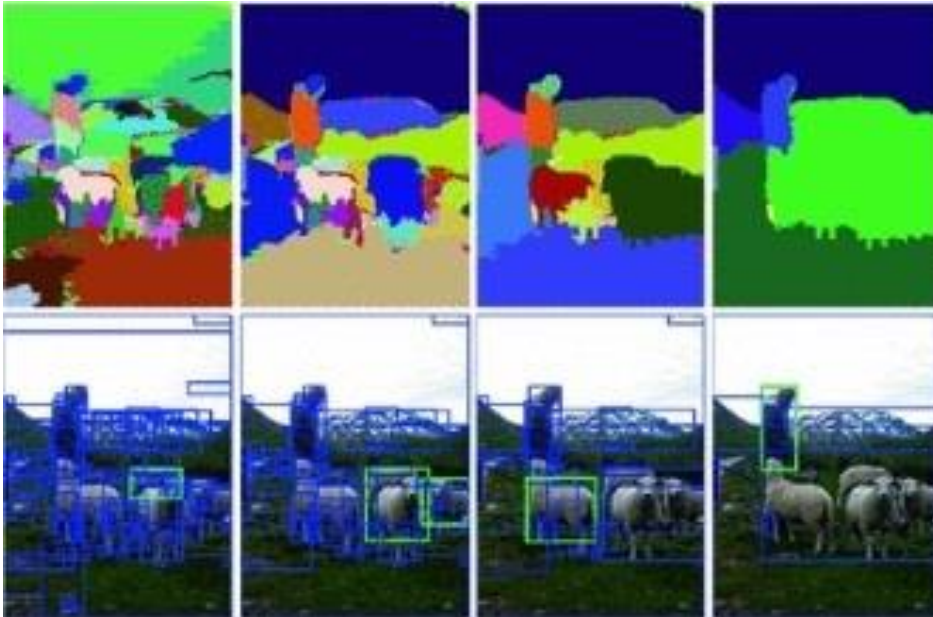


R-CNN

REGION PROPOSAL

Approach) 인근 픽셀 유사도 기반 클러스터링을 통해 객체가 있을만한 영역을
자동으로 찾아주는 비딤러닝 Region proposal 알고리즘 사용

Ex) Selective search – 2000여개 region proposal 제공, 약 2sec/image in CPU



R-CNN

Region-based CNN (CVPR 2014)

Input
image



Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.



R-CNN

Region-based CNN (CVPR 2014)

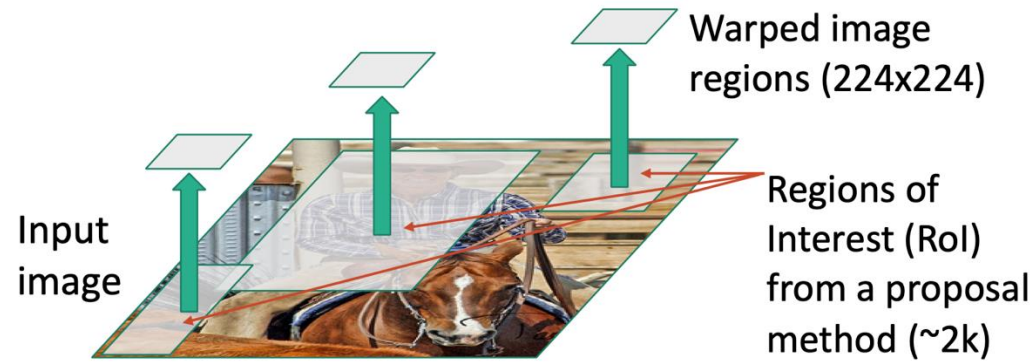


Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.



R-CNN

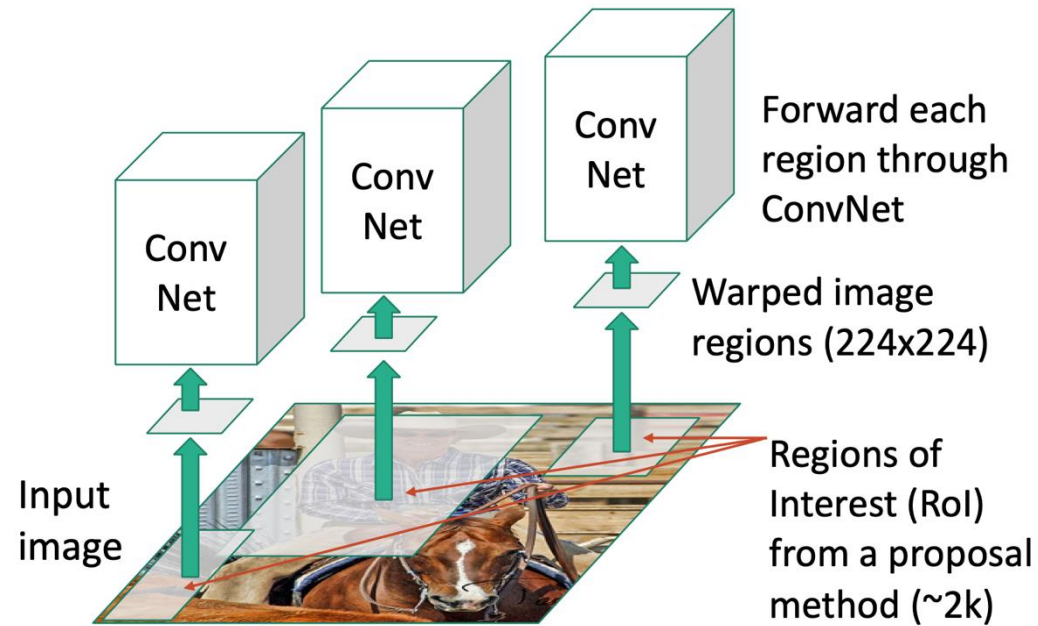
Region-based CNN (CVPR 2014)



Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

R-CNN

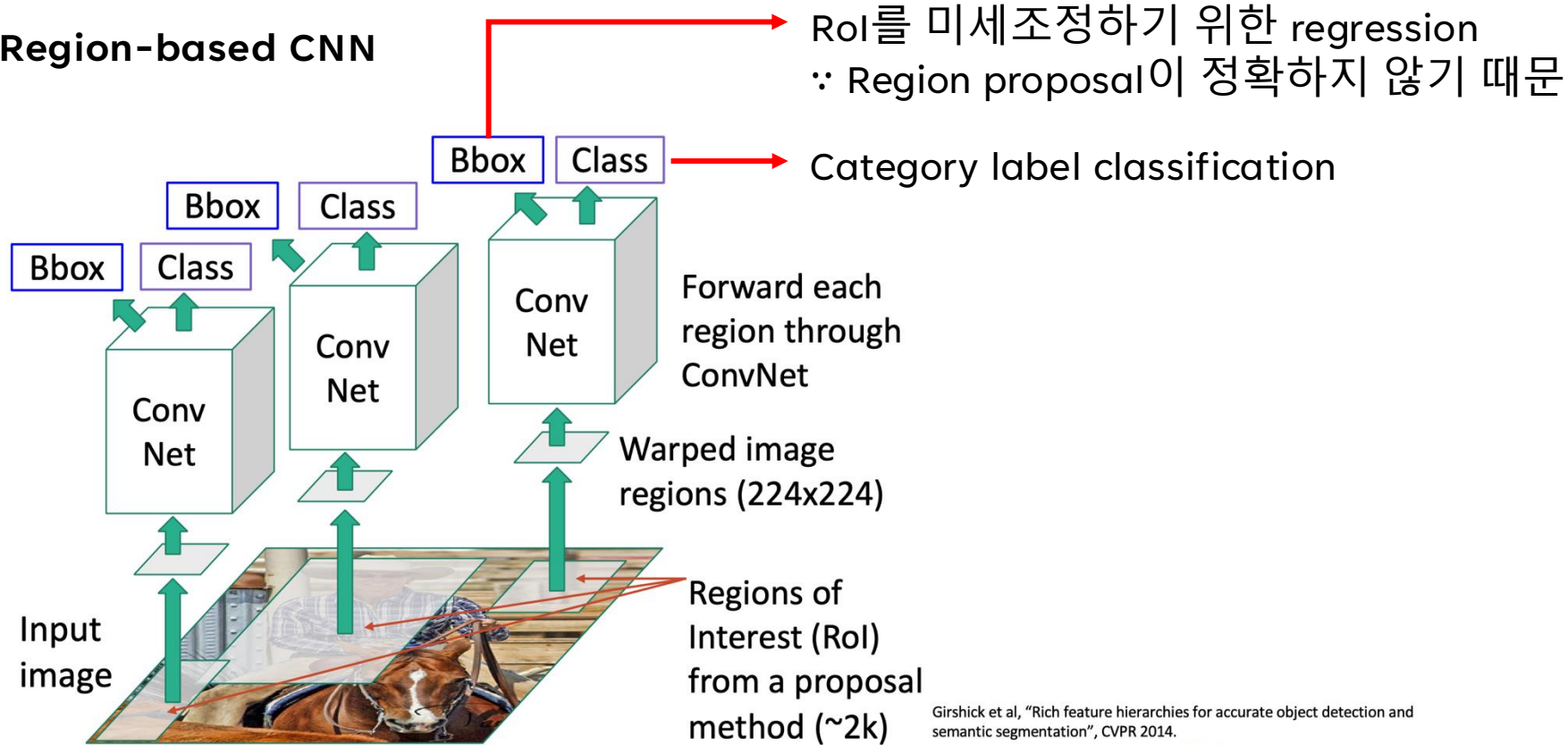
Region-based CNN (CVPR 2014)



Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

R-CNN

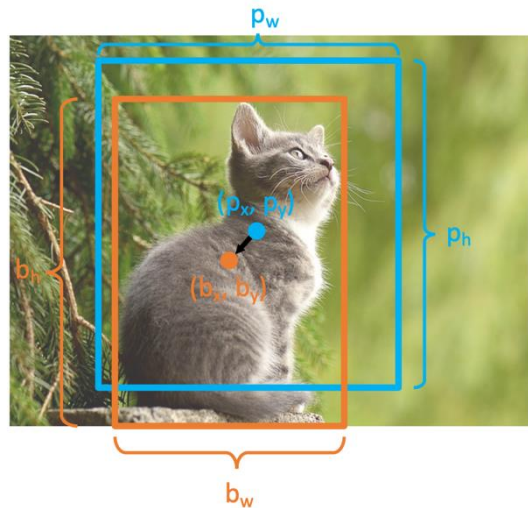
Region-based CNN



Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

BOX REGRESSION

Bounding box 좌표를 처음부터 예측 x, Region proposal 활용
Region proposal을 미세조정하기 위한 값을 모델이 출력하도록 함



Consider a **region proposal** with center (p_x, p_y) , width p_w , height p_h

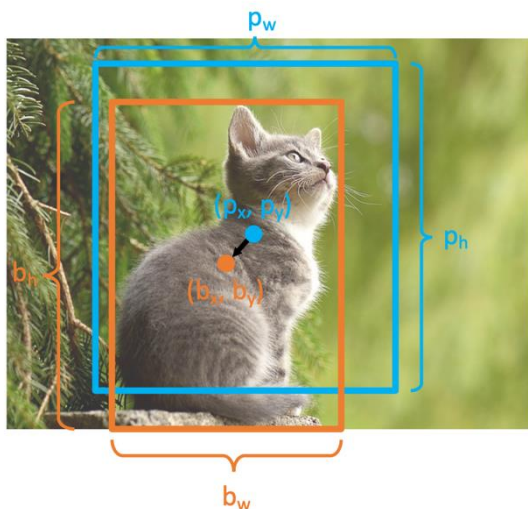
Model predicts a **transform** (t_x, t_y, t_w, t_h) to correct the region proposal

The **output box** is defined by:

$$\begin{aligned} b_x &= p_x + p_w t_x && \text{Shift center by amount relative to proposal size} \\ b_y &= p_y + p_h t_y \\ b_w &= p_w \exp(t_w) && \text{Scale proposal; exp ensures that scaling factor is } > 0 \\ b_h &= p_h \exp(t_h) \end{aligned}$$

BOX REGRESSION

Bounding box 좌표를 처음부터 예측 x, Region proposal 활용
Region proposal을 미세조정하기 위한 값을 모델이 출력하도록 함



Consider a **region proposal** with center (p_x, p_y) , width p_w , height p_h

Model predicts a **transform** (t_x, t_y, t_w, t_h) to correct the region proposal

The **output box** is defined by:

$$\begin{aligned} b_x &= p_x + p_w t_x && \text{Shift center by amount relative to proposal size} \\ b_y &= p_y + p_h t_y \\ b_w &= p_w \exp(t_w) && \text{Scale proposal; exp ensures that scaling factor is } > 0 \\ b_h &= p_h \exp(t_h) \end{aligned}$$

정답 bounding box, region proposal 좌표 통해
학습 objective 계산 & L2 loss로 학습

$$\begin{aligned} t_x &= (b_x - p_x) / p_w \\ t_y &= (b_y - p_y) / p_h \\ t_w &= \log(b_w / p_w) \\ t_h &= \log(b_h / p_h) \end{aligned}$$

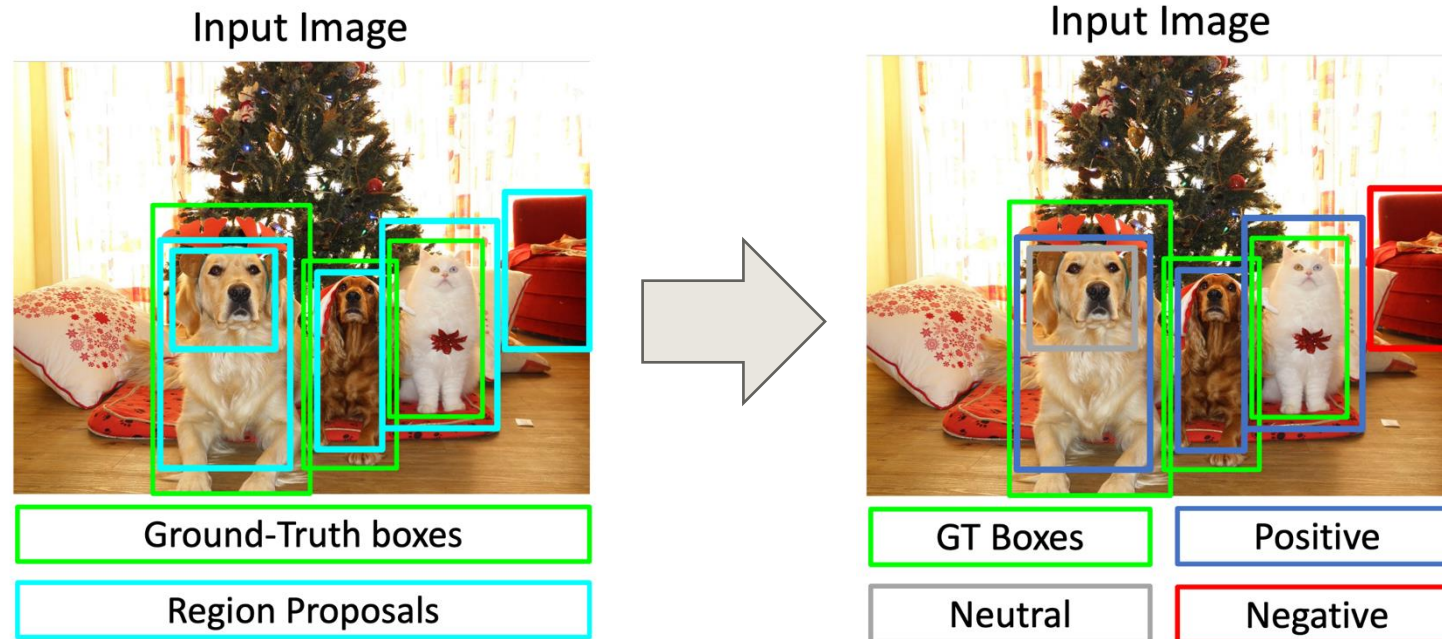


R-CNN TRAINING

Region proposal 개수가 아주 많으므로 모두 사용하지 x

Ground-truth box와의 IoU 계산해서 Positive, negative box만 학습에 활용

- Positive : 특정 GT box와의 IoU > 0.5
- Negative : 모든 GT box와의 IoU < 0.3



R-CNN TRAINING

각 region proposal을 224x224로 resize 후 CNN에 통과시킴

- Positive : Classification loss + Bounding box regression loss
- Negative : Classification loss 만 사용



An abstract graphic on a light gray background. Two thin, dark gray lines intersect. One line is nearly vertical, starting from the top center and extending towards the bottom right. The other line is nearly horizontal, starting from the top left and extending towards the right. The word "SUMMARY" is written in a bold, black, sans-serif font, positioned to the right of the intersection point.

SUMMARY

SUMMARY

1. Perception – overview
2. Object localization
3. Object detection
4. R-CNN



NEXT LECTURE

Perception (3) – 2D object detection - Continued



Two thin, dark gray lines intersect diagonally on a light gray background. One line slopes downwards from left to right, and the other slopes upwards from left to right, crossing it at approximately one-third of the way across the frame.

QUESTIONS?